



REHAB — Diseño del dataset y modelo de ML

Inteligencia artificial avanzada para la ciencia de
datos (Gpo 601)

Elisheba Hannaí Trejo Leyva | A01736967

02/09/2026

1. Objetivo

El objetivo de esta etapa del proyecto es construir una nueva representación del dataset **REHAB** que pueda ser utilizada para el entrenamiento de una técnica clásica de aprendizaje máquina.

A partir del análisis exploratorio de datos (EDA) realizado previamente, se busca transformar las señales temporales originales en una estructura más adecuada para un algoritmo de aprendizaje máquina. Posteriormente, se implementará manualmente un algoritmo capaz de aprender a partir de esta nueva representación, sin utilizar frameworks o bibliotecas que proporcionen el modelo ya implementado.

2. Dataset seleccionado

El dataset REHAB contiene dos conjuntos principales:

- Rehab_assessment
- Rehab_exercise

Para este proyecto se utilizará **Rehab_exercise**, ya que contiene los registros correspondientes a las diferentes actividades de rehabilitación analizadas previamente durante el EDA.

Dentro de Rehab_exercise se encuentran dos versiones de los datos:

- d01_raw_data: datos originales.
- d02_processed_data: datos preprocesados.

Para la construcción del nuevo dataset se utilizará **d02_processed_data**. La consigna permite trabajar tanto con los datos originales como con los preprocesados y, dado que el objetivo de esta etapa es la extracción de características y el entrenamiento de un modelo clásico, se decidió partir de la versión previamente procesada.

3. Inspección de la estructura original

Los registros se encuentran almacenados en archivos NumPy con extensión .npy.

Como primera inspección se comparó el archivo 000_1.npy en sus versiones raw y procesada.

En ambos casos se obtuvo:

(232, 880, 6)

Por lo tanto, el procesamiento realizado previamente **conserva las dimensiones originales del archivo**.

Los rangos observados fueron:

Versión	Valor mínimo	Valor máximo
Raw	-313.428	189.207
Procesada	-320.8824	145.6909

Estos resultados muestran que el procesamiento modifica los valores de las señales, aunque no altera las dimensiones de los datos.

Pendiente de verificación: actualmente se interpreta la primera dimensión como ejecuciones o muestras independientes. Antes de definir definitivamente las particiones del dataset, se verificará esta interpretación junto con el significado de los identificadores _1 y _2 de los archivos.

4. Exclusión de la actividad 014

Durante el EDA se identificó un comportamiento problemático en los registros correspondientes a la actividad 014.

Siguiendo la recomendación del profesor, esta actividad será excluida de la construcción del nuevo dataset para evitar que dichas anomalías afecten el entrenamiento del modelo.

Por lo tanto, se conservarán las actividades:

000–013,015

para un total de **15 clases**.

5. Propuesta de nueva estructura

El dataset original contiene series temporales extensas. Para utilizar una técnica clásica de aprendizaje máquina, se decidió transformar cada muestra en un **vector de características**.

Para cada una de las seis señales se extraen cuatro características descriptivas:

- **Media (mean):** representa el nivel promedio de la señal.
- **Desviación estándar (std):** representa su variabilidad.
- **Mínimo (min):** representa el valor inferior alcanzado.
- **Máximo (max):** representa el valor superior alcanzado.

De esta manera:

6 señales x 4 características = 24 características

A estas características se añadirá la **etiqueta de actividad** correspondiente a cada observación.

5.1 Verificación inicial de la extracción

Se realizó una prueba inicial utilizando una muestra del archivo 000_1.npy perteneciente a d02_processed_data. El archivo completo presenta dimensiones (232, 880, 6), mientras que una muestra individual presenta dimensiones (880, 6).

Al aplicar la extracción de características sobre esta muestra se obtuvo correctamente un vector de **24 características**, correspondientes a la media, desviación estándar, mínimo y máximo de cada uno de los seis canales.

Esta prueba permitió verificar el funcionamiento de la estrategia de extracción antes de aplicarla al conjunto completo de datos.

6. Partición de los datos para entrenamiento

Una vez generado el nuevo dataset de características, compuesto por **4,257 observaciones y 25 columnas** (24 características y la etiqueta **activity**), se realizó una partición de los datos antes de cualquier proceso de normalización o entrenamiento.

Se utilizó una división de aproximadamente **80 % para entrenamiento y 20 % como conjunto reservado**, obteniendo:

- **Training:** 3,400 observaciones.
- **Conjunto reservado:** 857 observaciones.

La partición se realizó de manera **estratificada por actividad**, separando individualmente las observaciones correspondientes a cada clase antes de unirlos nuevamente. De esta forma, las distintas actividades permanecen representadas en ambos conjuntos aproximadamente en la misma proporción que en el dataset generado.

Además, se utilizó una semilla aleatoria fija (42) para permitir que la partición pueda reproducirse al ejecutar nuevamente el programa.

6.1 Prevención de data leakage

La partición se realizó **antes de cualquier normalización de las características**. Esta decisión busca evitar *data leakage*, es decir, impedir que durante el entrenamiento se utilice indirectamente información proveniente de datos reservados.

Por esta razón, los parámetros necesarios para normalizar las características —como la media y la desviación estándar— serán calculados **exclusivamente utilizando el conjunto de entrenamiento**.

Posteriormente, cuando sea necesario transformar otros datos, se utilizarán los mismos parámetros obtenidos del conjunto de entrenamiento, sin recalcularlos a partir de los datos reservados.

6.1 Distribución de las clases

Para verificar la distribución de las observaciones entre las diferentes actividades, se generó una visualización del número de muestras correspondiente a cada clase. Se observa que las actividades presentan cantidades de observaciones relativamente similares, aunque existen algunas diferencias, destacando la actividad 007 como la de mayor número de muestras. La actividad 014 fue excluida del nuevo dataset de acuerdo con las decisiones tomadas durante el preprocesamiento.

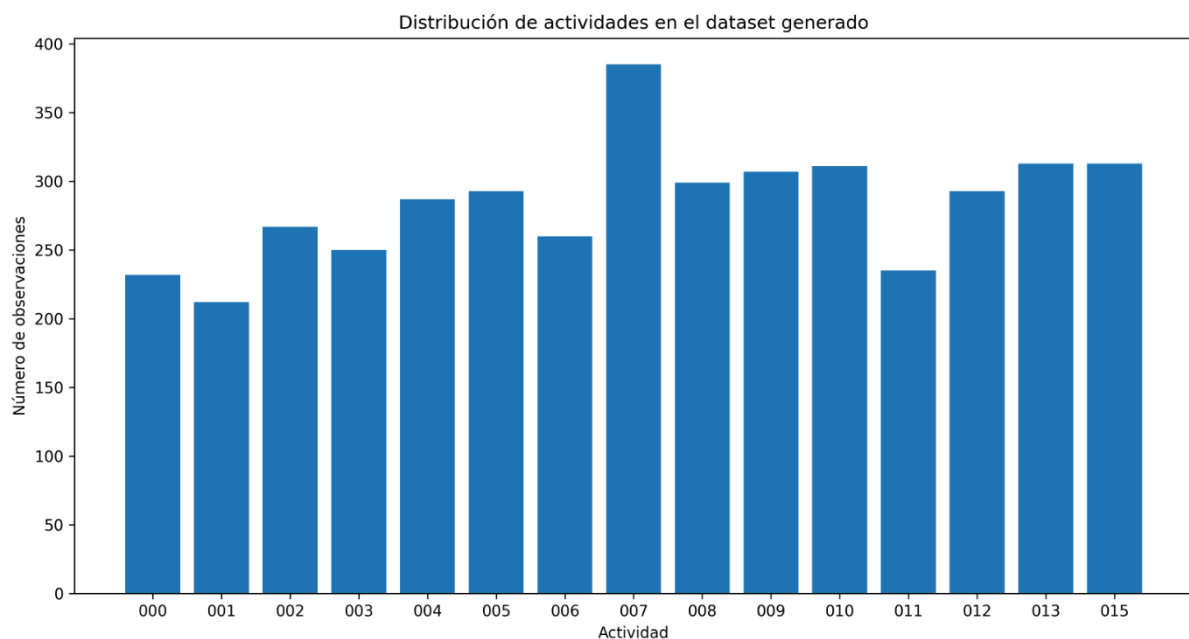


Figura 1. Distribución de actividades en el dataset generado.

Asimismo, se comparó la distribución de las actividades después de realizar la partición estratificada. Como se observa en la siguiente figura, tanto el conjunto de entrenamiento como el conjunto reservado mantienen un patrón de distribución similar entre las clases. Esto permite conservar la representación de las 15 actividades en ambos subconjuntos.

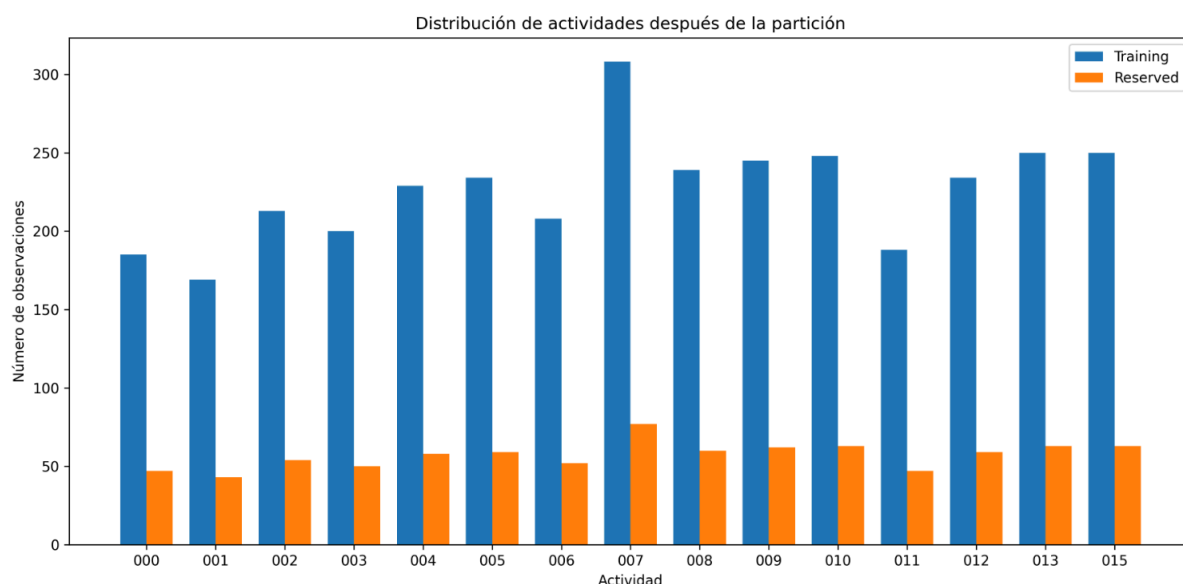


Figura 2. Distribución de actividades en los conjuntos de entrenamiento y reservado.

7. Normalización de los datos de entrenamiento

Debido a que el algoritmo seleccionado se basa en distancias entre observaciones, fue necesario estandarizar las características para evitar que aquellas con escalas numéricas mayores tuvieran una influencia desproporcionada.

Se utilizó una normalización mediante **Z-score**, definida como:

$$Z = \frac{x - \mu}{\sigma}$$

donde la media (μ) y la desviación estándar (σ) fueron calculadas **exclusivamente a partir del conjunto de entrenamiento**. Esta decisión permite evitar fuga de información (*data leakage*), ya que el conjunto reservado no interviene en la estimación de los parámetros utilizados durante el entrenamiento.

Como resultado, las características del conjunto de entrenamiento presentan aproximadamente una media de 0 y una desviación estándar de 1.

Los parámetros de normalización fueron almacenados por separado para permitir que, posteriormente, nuevos datos sean transformados utilizando exactamente las mismas estadísticas obtenidas del conjunto de entrenamiento.

8. Implementación del modelo KNN

Para realizar la clasificación de las actividades se implementó manualmente el algoritmo **K-Nearest Neighbors (KNN)**, sin utilizar frameworks o bibliotecas de aprendizaje máquina que proporcionaran el algoritmo ya implementado.

El modelo utiliza como entrada el conjunto de entrenamiento previamente generado y normalizado, compuesto por **3400 muestras y 24 características**. La variable objetivo corresponde a la actividad realizada, considerando 15 clases diferentes: 000 a 013 y 015. La actividad 014 permanece excluida del análisis.

Para esta primera implementación se utilizó $k = 5$, por lo que la predicción de una nueva muestra se realiza buscando sus cinco observaciones más cercanas dentro del conjunto de entrenamiento. La distancia entre observaciones se calcula mediante **distancia euclidiana** sobre las 24 características normalizadas.

Una vez identificados los vecinos más cercanos, la clase predicha corresponde a la clase con mayor frecuencia entre estos vecinos.

9. Prueba inicial del modelo

Después de implementar el algoritmo, se integró el modelo con el conjunto de datos real de REHAB mediante `main.py`.

El programa carga las **3400 muestras de entrenamiento**, separa las 24 características de la etiqueta de actividad y entrena el modelo KNN. Posteriormente, se realizaron predicciones sobre algunas muestras para comprobar el funcionamiento completo del algoritmo.

```
=== DATOS DE ENTRENAMIENTO ===
Muestras: 3400
Características: 24
Clases: ['000', '001', '002', '003', '004', '005', '006', '007', '008', '009', '010', '011', '012', '013', '015']

=== MODELO ===
KNN creado con k = 5
Modelo entrenado correctamente.

=== PREDICCIONES DE PRUEBA ===
Muestra 1: Real = 005 | Predicción = 004
Muestra 2: Real = 012 | Predicción = 012
Muestra 3: Real = 011 | Predicción = 011
Muestra 4: Real = 007 | Predicción = 007
Muestra 5: Real = 001 | Predicción = 002
```

Estos resultados permiten verificar que el algoritmo **ya es capaz de recibir observaciones reales del dataset y generar una clasificación**. En esta etapa, las predicciones corresponden únicamente a una prueba inicial sobre los datos de entrenamiento y **no deben interpretarse todavía como una evaluación de la capacidad de generalización del modelo**.

10. Accuracy de entrenamiento

Como verificación adicional del aprendizaje del algoritmo, se calcularon las predicciones para las **3400 muestras del conjunto de entrenamiento** y se compararon con sus etiquetas reales.

El modelo KNN implementado manualmente, utilizando $k = 5$, clasificó correctamente **2756 de las 3400 muestras**, obteniendo:

Accuracy de entrenamiento = 81.06 %

Este resultado indica que el modelo logra identificar correctamente una parte importante de las actividades utilizando las 24 características extraídas y normalizadas.

Sin embargo, este valor corresponde exclusivamente al **conjunto de entrenamiento**, por lo que no representa todavía el desempeño del modelo sobre datos no utilizados durante el entrenamiento ni su capacidad de generalización. El conjunto reservado se mantiene separado para evitar *data leakage* y permitir una evaluación posterior.

```
=== DATOS DE ENTRENAMIENTO ===
Muestras: 3400
Características: 24
Clases: ['000', '001', '002', '003', '004', '005', '006', '007', '008', '009', '010', '011', '012', '013', '015']

=== MODELO ===
KNN creado con k = 5
Modelo entrenado correctamente.

=== PREDICCIONES DE PRUEBA ===
Muestra 1: Real = 005 | Predicción = 004
Muestra 2: Real = 012 | Predicción = 012
Muestra 3: Real = 011 | Predicción = 011
Muestra 4: Real = 007 | Predicción = 007
Muestra 5: Real = 001 | Predicción = 002

=== ACCURACY DE ENTRENAMIENTO ===
Predicciones correctas: 2756 de 3400
Accuracy de entrenamiento: 0.8106
Accuracy de entrenamiento: 81.06%
```